
OpenRECAP: A Lightweight Self Improving Small Vision-Language-Action Model

Gustavo Garcia
UC San Diego
gag009@ucsd.edu

Ganesh Kumarappan
UC San Diego
gkumarappan@ucsd.edu

Parth Trivedi
UC San Diego
patrivedi@ucsd.edu

Rylen Sabhlok
UC San Diego
rsabhlok@ucsd.edu

Siddharth Maddikayala
UC San Diego
smaddikayala@ucsd.edu

Pranav Singh
UC San Diego
prs007@ucsd.edu

Abstract

Self improving robot policies that refine their behavior from autonomously collected experience have been demonstrated most convincingly on large, closed source vision-language-action (VLA) models that require substantial compute. Our work explores whether similar performance can be achieved on a *small, open source* VLA under a single GPU budget. We implement RECAP, the offline advantage conditioned procedure proposed by the authors of $\pi^*0.6$, on top of SmolVLA, and adapt it to the small model. We first replace RECAP’s large VLM value head with a lightweight state based MLP, label trajectory successes manually in a MuJoCo simulation environment, and fine tune the policy on the advantage relabeled dataset each iteration. Within this experiment, we isolate the effect of *reward signal design*, spending extra time discussing the impact of diverse reward structures. And finally, we found that the choice of reward signal is obvious; a sparse success only reward without regularization causes the policy to suffer catastrophic forgetting (from 50% to under 2% success), while a shaped dense reward improves success to a peak of 62%, with the gains concentrated at out of distribution object positions where the behavior cloning baseline fails. We implement a KL penalty toward the demonstration policy that stabilizes training but can also cap the achievable improvement. Our results show that RECAP style offline RL *does* transfer to smaller models, but only under careful reward and regularization design.

1 Introduction

Vision-language-action (VLA) models have rapidly become the dominant trend for generalist robot manipulation, mapping camera observations and natural language instructions directly to motor actions (1; 2; 3). Rather than relying solely on human demonstrations, they refine their own behavior by learning from conditioned experience. However, the models that have notably achieved this feat, such as $\pi^*0.6$ (4), are closed source, compute heavy, and difficult to reproduce. As a result, this method of robot learning remains out of reach for many researchers, students, and roboticists, who lack access to the model weights and the hardware budget to run them.

This inaccessibility is not only a matter of model scale, as most offline reinforcement learning (RL) techniques have been validated almost exclusively on large, proprietary policies. And thus, it remains an open question whether the same techniques transfer to **small, open source VLAs under a limited compute budget**, or whether they require extended modification to function in similar setups. Furthermore, the design of reward signals for offline reinforcement learning is still in its infancy, or in other words, how the success of an autonomous trajectory should be scored, and how aggressively

the policy should be allowed to move away from its demonstration behavior, are largely unexplored for compact models.

A central technical obstacle lies in the action representation itself. We study **SmolVLA** (5), a lightweight open source VLA that runs on a single GPU, pairing a SigLIP vision encoder and a SmolLM2 language backbone, with a *flow matching* action expert that predicts action chunks by iterative denoising. The problem is flow matching policies do not expose the action log probabilities that standard policy gradient methods such as PPO and SAC require (6; 7), making those algorithms mathematically incompatible with this class of model. **RECAP** (REinforcement learning with Corrective Advantage conditioned Policy optimization), the offline procedure underlying $\pi^*0.6$, accommodates this, where instead of differentiating through action probabilities, it labels each autonomously collected trajectory according to whether the task succeeded, estimates an advantage against a learned value function, and fine tunes the policy with supervised objectives that reward positive actions. This method requires no log probabilities nor is any live environment interaction required during the fine tuning step.

In this work we test whether this architecture survives the transition to the small model, single GPU setting. We implement RECAP on top of SmolVLA and evaluate it in a reproducible MuJoCo environment, including a SO-101 arm performing a cube pick and place task, in which a behavior cloned policy can be rolled out, scored, and fine tuned entirely offline. Within this environment, we isolate the effect of *reward signal design*, varying both the density of the reward and the strength of the regularization that keeps the policy pertaining to its demonstration behavior. In other words, we focus on the question:

Can RECAP-style offline RL improve a small open-source VLA, and how does reward signal design affect the outcome?

Contributions. Our work makes the following contributions:

- **An open, single GPU implementation of RECAP for a small VLA.** We adapt an offline, advantage-conditioned RL procedure which was previously demonstrated only on large, closed-source models, to SmolVLA, noting any modifications required to make it compatible with a flow-matching action expert that exposes no action log-probabilities.
- **A reproducible MuJoCo manipulation environment.** We build a simulated SO-101 cube pick and place environment with a behavior-cloning (Imitation Learning) baseline, designed so that the full self-improvement loop, rollout, reward labeling, value estimation, advantage computation, and fine-tuning, runs end-to-end on common hardware.
- **An analysis of reward signal design.** We ablate sparse versus dense rewards and the presence or absence of policy regularization, characterizing when offline RL improves a compact VLA versus when it collapses, before finally providing practical guidance for applying these methods outside without a surplus of compute resources.

2 Related Work

$\pi^*0.6$ and advantage-conditioned offline RL. Our work is most directly inspired by $\pi^*0.6$ (4), which introduces RECAP to make a flow-matching VLA self-improving from its own successes and failures. $\pi^*0.6$ demonstrates the approach at scale but relies on closed weights and custom robot hardware, placing it out of reach of most researchers. We retain its core advantage-conditioned recipe while targeting a small, open model on a single GPU.

Online RL for flow-matching policies. A complementary line of work, π -RL (8), develops *online* RL for flow-matching VLAs through estimators such as Flow-Noise and Flow-SDE that recover usable gradients from the denoising process. Unlike RECAP, these methods require live environment interaction during training. We instead remain fully offline: the policy never interacts with the environment during the fine-tuning step, only during periodic rollout collection.

Reinforcement fine-tuning of VLAs. VLA-RFT (9) performs reinforcement fine-tuning of VLAs using a learned world model together with GRPO-style optimization. This shares our goal of

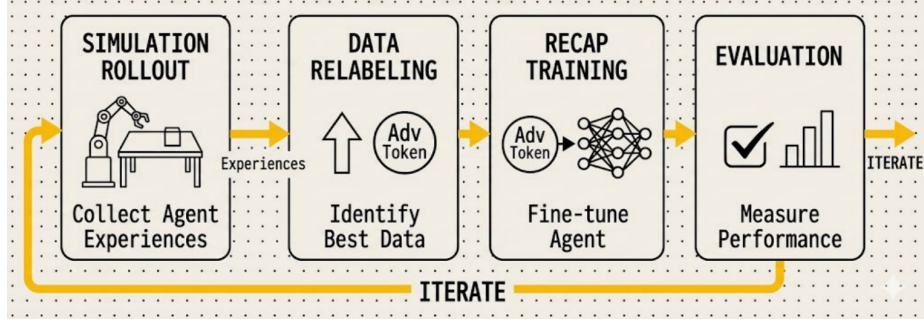


Figure 1: The RECAP self-improvement loop: roll out the behavior-cloned policy in simulation, relabel the collected experience by advantage, fine tune SmolVLA on the best data, evaluate, and iterate. The loop runs entirely on a single GPU.

improving a VLA beyond its demonstration data, but depends on training a world model; our variant replaces any large auxiliary model with a lightweight state-based value function.

SmolVLA and open VLAs. SmolVLA (5) is a compact ($\sim 450\text{M}$ -parameter) open-source flow-matching VLA that can be trained and deployed on a single GPU and runs in both simulation and on real SO-101 hardware. It provides the open, accessible substrate on which we build; to our knowledge, ours is among the first open implementations of advantage-conditioned offline RL on a model of this scale.

3 Method

3.1 Background: SmolVLA and the RECAP objective

SmolVLA processes camera frames through a SigLIP vision encoder, applying a pixel-shuffle operation to compress each frame to 64 visual tokens for faster inference. These visual tokens, language tokens, and proprioceptive state tokens are fused by a SmolLM2 decoder, of which only the first half of the layers are used (layer skipping). A flow-matching *action expert* then predicts action chunks by iterative denoising, attending to the backbone context through interleaved cross- and self-attention over action queries.

Because the action expert is a flow-matching model, it does not expose action log-probabilities, ruling out PPO/SAC-style updates. RECAP avoids this by treating policy improvement as supervised fine-tuning on a *reabeled* dataset. Given autonomously collected trajectories, each transition is assigned a return, a value baseline $V(s)$ is fit, and the advantage $A(s, a) = R - V(s)$ is used to mark high-advantage transitions as “positive.” The policy is then fine tuned, behavior cloning-style, to prefer the positive-labeled actions.

3.2 General workflow

Our self-improvement loop (shown in Figure 1) repeats four stages: (1) **Simulation rollout**: roll out the current policy in simulation to collect agent experiences; (2) **Data relabeling**: score trajectories and identify the best data, attaching an advantage token; (3) **RECAP training**: fine-tune the agent on the relabeled dataset; and (4) **Evaluation**: measure performance. This loop then continues to iterate, with each round optimizing the improved policy using the previous round’s policy.

3.3 Simulation environment and task

We use a MuJoCo simulation of the SO-101 5-DOF arm with a parallel jaw gripper on a tabletop scene containing a free floating cube and an open-top bin (Figure 2). The policy receives a front camera and a wrist camera RGB stream together with the 6-dimensional joint state, and joint commands

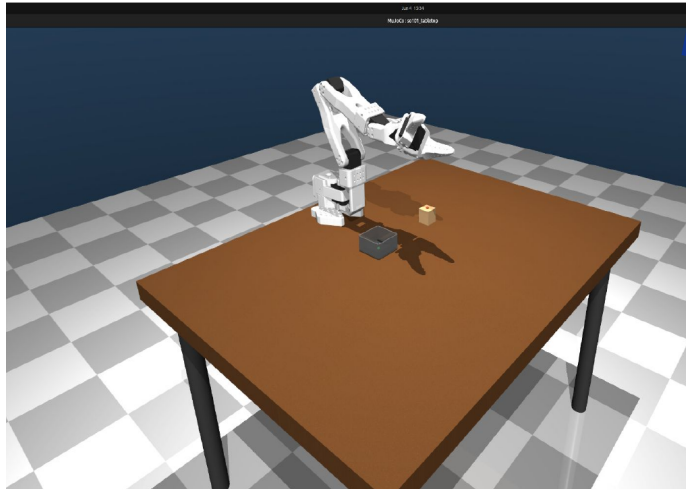


Figure 2: The MuJoCo SO-101 cube pick and place environment: a 5-DOF arm must place the cube into the bin. The same interface drives both simulation and real SO-101 hardware.

normalized at 30 Hz. A trajectory is counted as a success when the cube comes to rest inside the bin by the end of the episode. To push for VLA generalization, the cube is initialized at one of three positions: the training position and two 3 cm offsets (left and forward).

3.4 Data collection

The behavior-cloning seed policy is trained on human teleoperation demonstrations. A physical SO-101 leader arm is mirrored into the MuJoCo simulation, and observations and actions are recorded as a LeRobot-format dataset. SmolVLA is then fine-tuned on these demonstrations to produce the behavior-cloning (BC) checkpoint that initializes every RECAP run.

3.5 Our RECAP variant for the single-GPU regime

We adapt RECAP to the small-model setting with three changes (Figure 1):

1. **Sim rollouts with an in-loop oracle.** We roll out the policy on the MuJoCo cube bin task and label each trajectory success or failure in-loop using a scripted inverse-kinematics oracle, removing the need for a separate reward-classification pass.
2. **A lightweight value function.** RECAP’s value baseline is normally a large VLM value head ($\sim 670\text{M}$ parameters). We replace it with a small MLP trained on $(state, return)$ pairs, which removes the dominant memory and compute cost of the value-estimation step and lets the entire loop fit on a single GPU.
3. **Advantage-relabeled fine-tuning.** The top $\sim 30\%$ of transitions by advantage receive a “positive” token, and SmolVLA is re-fine-tuned BC-style on the relabeled dataset each iteration.

Reward signal design. We study two axes of the reward signal. The first is *density*: a **sparse** reward that fires only on task success, versus a **dense** reward shaped by task progress (e.g., proximity of the cube to the bin). The second is *regularization*: whether or not a KL penalty is applied to keep the fine-tuned policy close to the BC reference. Crossing these axes yields four conditions, $\{\text{sparse}, \text{dense}\} \times \{\text{no-KL}, \text{KL}\}$, which we compare against the frozen BC baseline.

Table 1: Behavior-cloning baseline success rate by cube position (20 episode evaluation). The BC policy is perfect at the training position but fails completely at both 3 cm offsets. It has memorized the demonstration position.

Cube position	Offset	Success rate
pos 0	training position	7/7 (100%)
pos 1	3 cm left	0/7 (0%)
pos 2	3 cm forward	0/6 (0%)
Overall		7/20 (35%)

Table 2: Overall success rate (%) across RECAP iterations. Dense no-KL is the only condition with a clear net improvement (peak 62% at iteration 5); sparse no-KL collapses, and both KL-regularized conditions are stable but capped. Baseline is the frozen BC policy.

Iter	Baseline	Sparse no-KL	Dense no-KL	Sparse +KL	Dense +KL
0	50.0	50.0	50.0	50.0	50.0
1	n/a	40.0	49.7	48.3	49.0
2	n/a	31.7	52.0	48.3	50.0
3	n/a	21.7	55.3	46.7	51.3
4	n/a	11.7	58.3	46.7	51.7
5	n/a	6.7	62.0	46.7	52.0
6	n/a	3.3	60.0	45.0	51.7
7	n/a	1.7	56.7	45.0	50.7
8	n/a	1.7	53.3	45.0	50.0

4 Experiments

4.1 Setup

All runs start from the same BC checkpoint and perform RECAP for eight iterations. Each condition is evaluated by rolling the policy out across the three cube positions and reporting success rate (%). We report the BC baseline as a reference and the four reward conditions across iterations.

4.2 The behavior-cloning baseline overfits to the training position

Before applying RECAP, we characterize the BC policy. Table 1 reports an independent evaluation of the BC checkpoint over 20 episodes with the cube cycled through the three positions. The policy succeeds at every trial at the *training* position but never succeeds at either 3 cm offset, a clear case of memorizing the demonstration distribution rather than learning a position invariant skill. This brittleness is precisely the gap RECAP is meant to close.

4.3 Reward signal design determines success

Table 2 reports overall success rate across RECAP iterations for all four reward conditions, with the full per-position trajectory shown in Figure 3. The dense no-KL condition improves monotonically over the baseline to a peak of 62% at iteration 5, whereas the sparse no-KL condition degrades from 50% to under 2%. Both KL-regularized conditions remain stable across all eight iterations but show only marginal movement: dense+KL plateaus near 52% and sparse+KL settles slightly below baseline at $\sim 45\%$. Table 3 breaks the best configuration (dense no-KL at iteration 5) down by cube position, showing that its improvement is concentrated at the out-of-distribution offsets.

4.4 Discussion

Our results yield three findings about offline RL in the small-model regime.

Sparse rewards induce catastrophic forgetting. Without a dense signal, SmolVLA collapses to near zero success by iteration 8 (Table 2). With only a success or failure label, the advantage

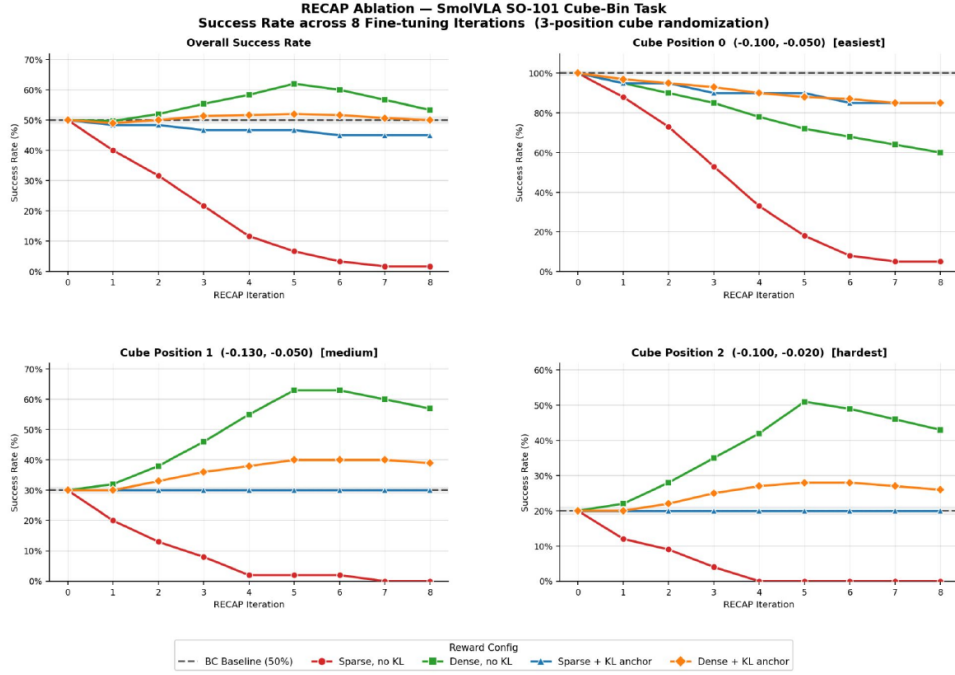


Figure 3: Success rate across 8 RECAP iterations for all four reward configurations. Top-left shows the overall success rate; the remaining panels break it down by cube position (easiest, medium, hardest). Dense no-KL (green) is the only configuration that clearly improves over the BC baseline, and the improvement is largest at the hardest, out-of-distribution position. Sparse no-KL (red) collapses across all positions, while both KL-anchored conditions remain stable but capped near baseline.

Table 3: Per-position success rate (%): BC baseline versus each reward condition at its representative iteration (best for dense no-KL, iteration 5; final for the others). Dense no-KL more than doubles success at both out of distribution offsets.

Cube position	Baseline	Sparse no-KL	Dense no-KL	Sparse +KL	Dense +KL
pos 0 (training)	100	5	72	90	88
pos 1 (3 cm left)	30	0	63	30	40
pos 2 (3 cm forward)	20	0	51	20	28
Overall	50	1.7	62	46.7	52

estimates are too noisy for a model with limited pretraining: each round of re-fine-tuning supervises the policy on a large fraction of mislabeled “positive” actions, and the policy progressively forgets the competent behavior it inherited from behavior cloning. Capacity that would absorb this noise in a large model is simply not available here.

Dense rewards rescue smaller models. A shaped dense reward supplies the value function with usable signal from the very first iteration, converting the sparse collapse into a monotonic improvement that peaks at 62% (iteration 5). Notably, this inverts the usual trade-off: in *larger* models a dense, shaped reward is a liability because it invites reward hacking, whereas for a small, lightly-pretrained policy the same density is what makes advantage estimation tractable at all. Reward shaping is therefore not a fixed good or bad; its value depends on model scale.

The hardest position shows the widest gap. The benefit of dense rewards is clearest exactly where the baseline is weakest. At the most difficult cube position (pos 2), the BC policy reaches only 20%, whereas dense no-KL climbs to 51% by iteration 5 (Table 3), more than doubling success at the out-of-distribution offset. RECAP’s gains are thus gains in spatial generalization, directly addressing the memorization failure of Table 1.

The role of KL regularization. The KL penalty toward the BC reference is a reliable safeguard against the sparse collapse, but at the strength tested it also holds the policy too close to its demonstration behavior to realize the larger gains of dense no-KL (Table 2). Because regularization is what suppresses reward hacking in large models but caps improvement in small ones, an annealed or adaptively weighted penalty is a natural direction for future work.

5 Limitations

Several limitations qualify our findings. **Compute.** Even with a relatively small 450M-parameter model, each RECAP iteration requires collecting enough rollouts to estimate advantages reliably, so a full dense versus sparse comparison still consumes significant GPU time; accessibility is improved but not free. **Sim-to-real gap.** Our policy is trained and evaluated entirely in MuJoCo; lighting, sensor noise, and physics mismatch mean these success rates may not transfer directly to a physical SO-101. **Constrained setup.** The decision to remain in simulation was itself forced by hardware limitations on recording real robot motion, which also restricted our task and environment design. **Pre-training gap.** Finally, a technique that performs well on a large closed-source model does not guarantee good performance on an open one: open and closed models of different sizes differ in their pretraining, and this gap is a confound we cannot fully control. We identify it as a key limiting factor for porting frontier self-improvement recipes to accessible models.

6 Conclusion

We presented an open, single-GPU implementation of RECAP for SmolVLA, built on a MuJoCo SO-101 cube pick and place environment constructed from scratch, in which RECAP’s large VLM value head is replaced by a lightweight state-based MLP and trajectory success is labeled in-loop by a scripted oracle. Studying reward signal design, we found that a shaped dense reward without KL regularization improves overall success from 50% to a peak of 62% on a single GPU within eight iterations, driven by large gains at out-of-distribution object positions, whereas a sparse reward causes catastrophic forgetting and KL regularization stabilizes training at the cost of capping gains. Our contributions are an open-source RECAP implementation, a reproducible simulation testbed, a systematic study of reward signal design in the small-model regime, and the identification of the open/closed pre-training gap as a limiting factor for transferring these methods. Together they show that self-improving robot learning, demonstrated so far only on large closed systems, can be made to work on small open models, accessible to everyone, given a reward signal designed for the low-capacity regime.

References

- [1] Ma, Y., Song, Z., Zhuang, Y., Hao, J., and King, I. (2024). A survey on vision-language-action models for embodied AI. *arXiv preprint arXiv:2405.14093*.
- [2] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al. (2023). RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*.
- [3] Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., and Finn, C. (2024). OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*.
- [4] Physical Intelligence, Amin, A., Aniceto, R., Balakrishna, A., Black, K., Conley, K., Connors, G., Darpinian, J., Dhabalia, K., DiCarlo, J., Driess, D., et al. (2025). $\pi_{0.6}^*$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*.

- [5] Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., et al. (2025). SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*.
- [6] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- [7] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the International Conference on Machine Learning (ICML)*. *arXiv preprint arXiv:1801.01290*.
- [8] Chen, K., Liu, Z., Zhang, T., Guo, Z., Xu, S., Lin, H., Zang, H., Li, X., Zhang, Q., Yu, Z., Fan, G., Huang, T., Wang, Y., and Yu, C. (2025). π_{RL} : Online RL fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*. *NeurIPS 2025*.
- [9] Li, H., Ding, P., Suo, R., Wang, Y., Ge, Z., Zang, D., Yu, K., Sun, M., Zhang, H., Wang, D., et al. (2025). VLA-RFT: Vision-language-action reinforcement fine-tuning with verified rewards in world simulators. *arXiv preprint arXiv:2510.00406*.
- [10] Todorov, E., Erez, T., and Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033.

A Technical appendices and supplementary material

Additional results, environment details, and training hyperparameters can be included here. The appendix does not count toward the page limit.